

Mathematical Engine Notes for the PRNG Casino Modules

1 Scope

This document explains the math behind the modules in this repository:

- `prng.py`: shared deterministic HMAC-SHA256 pseudo-random generator.
- `game_config.json` and `game_config.py`: editable game math settings.
- `cards.py`: deck construction and unbiased Fisher-Yates shuffling.
- `prng_crash_limbo.py`: crash-game multiplier generation.
- `slots.py`: weighted reels, paylines, scatters, and RTP calculation.
- `baccarat.py`: digital baccarat dealing, totals, draw rules, and settlement.
- `blackjack.py`: digital blackjack dealing, hand valuation, actions, dealer rules, and settlement.

The common design is:

(server seed, client seed, nonce) $\longrightarrow$ HMAC-SHA256 output $\longrightarrow$ game-specific outcome.

The server seed is private before the game. Its hash may be published first as a commitment:

$$C_S = \text{SHA256}(S),$$

where S is the server seed and C_S is the public commitment. After the game, revealing S lets a player verify that the seed matches the prior commitment.

2 Shared PRNG: `prng.py`

2.1 HMAC Byte Stream

The generator creates a deterministic byte stream from:

$$S = \text{server seed}, \quad C = \text{client seed}, \quad n = \text{nonce}, \quad i = \text{cursor}.$$

For each cursor value i , the message is:

$$M_i = C \parallel n \parallel i.$$

The digest block is:

$$B_i = \text{HMAC-SHA256}(S, M_i).$$

Each digest block is 32 bytes. The byte stream is:

$$B = B_0 \| B_1 \| B_2 \| \dots$$

The same inputs always produce the same bytes. Without the server seed, the stream is computationally unpredictable.

2.2 Code

```
message = f"{self.client_seed}:{self.nonce}:{self.cursor}".encode()
output += hmac.new(self.server_seed.encode(), message, hashlib.sha256).digest()
self.cursor += 1
```

The commitment helper is:

```
def server_seed_hash(server_seed: str) -> str:
    return hashlib.sha256(server_seed.encode()).hexdigest()
```

2.3 Unbiased Bounded Integers

Many games need a random integer:

$$X \in \{0, 1, \dots, m-1\}.$$

A naive method would be:

$$X = R \bmod m,$$

but this creates modulo bias when the sample space size is not divisible by m .

The module avoids this with rejection sampling. Let:

$$b = \max(1, \lceil \text{bitlength}(m)/8 \rceil).$$

The sample space is:

$$N = 2^{8b}.$$

The largest unbiased accepted range is:

$$L = N - (N \bmod m).$$

The generator draws:

$$R \in \{0, 1, \dots, N-1\}.$$

If $R \geq L$, the value is rejected and another value is drawn. If $R < L$, the result is:

$$X = R \bmod m.$$

Because L is divisible by m , every output in $\{0, \dots, m-1\}$ has exactly the same number of preimages.

2.4 Code

```
byte_length = max(1, math.ceil(upper_bound.bit_length() / 8))
sample_space = 1 << (8 * byte_length)
unbiased_limit = sample_space - (sample_space % upper_bound)

while True:
    value = int.from_bytes(self.random_bytes(byte_length), "big")
    if value < unbiased_limit:
        return value % upper_bound
```

3 Game Configuration: game_config.json

The repository now has an editable JSON config file for target RTPs and rule settings:

```
{
  "crash": {
    "target_rtp": 0.98,
    "instant_crash_modulus": 25
  },
  "slots": {
    "target_rtp": 0.96,
    "volatility": 1.0,
    "calibration_max_combinations": 1000000
  },
  "baccarat": {
    "target_rtp": 0.9876,
    "decks": 8,
    "banker_commission": 0.05,
    "tie_payout": 8.0
  },
  "blackjack": {
    "target_rtp": 0.995,
    "decks": 6,
    "dealer_hits_soft_17": false,
    "blackjack_payout": 1.5
  }
}
```

The values are decimals, so:

$$0.98 = 98\% \text{ RTP} .$$

For crash, `target_rtp` is used directly as the payout scale:

$$p = 100 \cdot \text{target_rtp}.$$

For baccarat and blackjack, RTP is primarily an output of rules and paytables. The `target_rtp` value documents the intended target, while rule settings such as commission, tie payout, blackjack payout, dealer behavior, and deck count are what actually change the realized RTP.

For slots, `target_rtp` can now be applied directly by calibrating a `SlotConfig`. The module first applies a volatility shape, computes the shaped exact RTP, then scales every line and scatter payout by:

$$\alpha = \frac{\text{target_rtp}}{\text{RTP}_{\text{shaped}}}.$$

After scaling, the exact RTP becomes:

$$\text{RTP}_{\text{final}} = \alpha \text{RTP}_{\text{shaped}} = \text{target_rtp}.$$

This works because the slot module's payout model is linear in paytable and scatter multipliers. Slot volatility is a numeric payout exponent:

$$v = \text{volatility}.$$

Before RTP calibration, each positive payout m becomes:

$$m' = m^v,$$

Values above 1 make large wins larger relative to small wins. Values below 1 compress large wins toward small wins. A value of 1 leaves payout shape unchanged. This changes payout variance while the later RTP calibration keeps the long-run average fixed.

3.1 Code

```
def target_rtp(game: str, default: float) -> float:
    value = float(setting(game, "target_rtp", default))
    if value <= 0 or value > 1:
        raise ValueError(f"{game} target_rtp must be between 0 and 1")
    return value
```

4 Cards and Shuffling: cards.py

4.1 Deck Construction

For one deck:

$$|\text{deck}| = 13 \cdot 4 = 52.$$

For d decks:

$$|\text{shoe}| = 52d.$$

Cards are rank-suit pairs:

$$\text{Card} = (\text{rank}, \text{suit}).$$

4.2 Fisher-Yates Shuffle

For a deck array A of length N :

for $i = N - 1, N - 2, \dots, 1$:

$j \leftarrow \text{random_int}(i + 1),$

$A_i \leftrightarrow A_j.$

Because `random_int` is unbiased, every physical permutation is equally likely:

$$\Pr(\text{specific physical order}) = \frac{1}{N!}.$$

For d decks, duplicate rank-suit labels exist, but the underlying shoe positions are still shuffled uniformly.

4.3 Code

```
def build_deck(decks: int = 1) -> Deck:
    return tuple(Card(rank, suit) for _ in range(decks) for suit in SUITS for rank in
RANKS)

def fisher_yates_shuffle(cards: Sequence[Card], rng: FairRng) -> Deck:
    shuffled = list(cards)
    for index in range(len(shuffled) - 1, 0, -1):
        swap_index = rng.random_int(index + 1)
        shuffled[index], shuffled[swap_index] = shuffled[swap_index], shuffled[index]
    return tuple(shuffled)
```

5 Crash Multiplier: prng_crash_limbo.py

5.1 Per-Game Server Seed

The crash module derives a per-game seed:

$$S_n = \text{SHA256}(S_0 \| : \| n).$$

Then it computes:

$$D = \text{HMAC-SHA256}(S_n, C \| : \| n \| : 0).$$

5.2 Instant Crash

The code checks the first 8 hex characters of the digest:

$$Y = \text{int}(D_{0:8}, 16).$$

This is a 32-bit value. With modulus $q = 25$, instant crash occurs when:

$$Y \bmod q = 0.$$

The exact finite probability is:

$$\Pr(\text{instant crash}) = \frac{\lfloor (2^{32} - 1)/25 \rfloor + 1}{2^{32}}.$$

This is approximately:

$$\Pr(\text{instant crash}) \approx \frac{1}{25} = 0.04.$$

5.3 Multiplier Formula

If no instant crash occurs, the module takes 13 hex characters:

$$h = \text{int}(D_{0:13}, 16).$$

Since one hex character is 4 bits:

$$13 \cdot 4 = 52,$$

so:

$$0 \leq h < e, \quad e = 2^{52}.$$

With house edge $r = 2\%$, the payout percent is:

$$p = 100 - r = 98.$$

The multiplier is:

$$M = \max\left(1, \frac{pe - h}{100(e - h)}\right).$$

As h approaches e , the denominator $e - h$ becomes small, creating rare large multipliers. Because of the $\max(1, \cdot)$ clamp, some non-instant-crash hashes also return exactly 1.00. The raw expression is at most 1 when:

$$\frac{pe - h}{100(e - h)} \leq 1.$$

For $p = 98$, this simplifies to:

$$h \leq \frac{2}{99}e.$$

5.4 Code

```
server_seed = derive_server_seed(initial_seed, game_number)
digest = hmac_sha256(server_seed, client_seed, nonce).hex()

if is_divisible(digest, INSTANT_CRASH_MODULUS):
    return 1.0

hash_int = int(digest[:13], 16)
max_hash_int = 2**52
payout_percent = 100 - HOUSE_EDGE_PERCENT

return max(1.0, (payout_percent * max_hash_int - hash_int) / (max_hash_int - hash_int) /
            100)
```

6 Slots: slots.py

6.1 Weighted Reels

A reel is a sequence:

$$R_k = (s_{k,0}, s_{k,1}, \dots, s_{k,L_k-1}),$$

where k is the reel index and L_k is the reel length.

The helper `expand_weighted_reel` converts weights into repeated symbols. If symbol a has weight w_a , then:

$$\Pr(a \text{ is selected on that reel}) = \frac{w_a}{\sum_b w_b}.$$

6.2 Code

```
def expand_weighted_reel(symbol_weights: Mapping[Symbol, int]) -> Reel:
    reel: list[Symbol] = []
    for symbol, weight in symbol_weights.items():
        reel.extend([symbol] * weight)
    return tuple(reel)
```

6.3 Reel Stops

For each reel k , the PRNG chooses:

$$x_k \in \{0, 1, \dots, L_k - 1\}.$$

The stop tuple is:

$$\mathbf{x} = (x_0, x_1, \dots, x_{K-1}).$$

Because each stop is selected with unbiased `random_int`:

$$\Pr(\mathbf{x}) = \prod_{k=0}^{K-1} \frac{1}{L_k}.$$

6.4 Visible Grid

For r visible rows:

$$G_{i,k} = R_k[(x_k + i) \bmod L_k].$$

6.5 Code

```
rng = FairRng(server_seed, client_seed, nonce)
stops = tuple(rng.random_int(len(reel)) for reel in config.reels)

def visible_grid(reels: Sequence[Reel], stops: Sequence[int], rows: int) -> Grid:
    columns = []
    for reel, stop in zip(reels, stops):
        columns.append(tuple(reel[(stop + row) % len(reel)] for row in range(rows)))
    return tuple(tuple(column[row] for column in columns) for row in range(rows))
```

6.6 Payline Evaluation

A payline is:

$$\ell = (\ell_0, \ell_1, \dots, \ell_{K-1}).$$

The symbols on the line are:

$$(G_{\ell_0,0}, G_{\ell_1,1}, \dots, G_{\ell_{K-1},K-1}).$$

For each pay symbol a , the module counts consecutive symbols from the left that are either a or the wild symbol W :

$$c_a = \max c \text{ such that } G_{\ell_i,i} \in \{a, W\} \text{ for } 0 \leq i < c.$$

The selected line win is:

$$\max_a \text{paytable}(a, c_a).$$

6.7 Code

```
for pay_symbol, payouts in config.paytable.items():
    count = 0
    for symbol in symbols:
        if symbol == pay_symbol or symbol == config.wild_symbol:
            count += 1
        else:
            break
    multiplier = payouts.get(count)
```

6.8 Scatter Evaluation

If a scatter symbol S is configured:

$$c_S = \sum_{i,k} \mathbf{1}(G_{i,k} = S).$$

If `scatter_pays( $c_S$ )` exists, it is added to the total multiplier.

6.9 Code

```
count = sum(symbol == config.scatter_symbol for row in grid for symbol in row)
multiplier = config.scatter_pays.get(count)
```

6.10 Total Multiplier and Payout

$$T = \sum_{\text{line wins } w} w + \text{scatter win.}$$

For bet B :

$$\text{payout} = BT.$$

6.11 Code

```
total_multiplier = sum(win.multiplier for win in line_wins)
if scatter_win is not None:
    total_multiplier += scatter_win.multiplier
payout = bet * total_multiplier
```

6.12 Exact and Sample RTP

Let:

$$\Omega = \prod_{k=0}^{K-1} \{0, 1, \dots, L_k - 1\}.$$

Then:

$$|\Omega| = \prod_{k=0}^{K-1} L_k.$$

Exact RTP is:

$$\text{RTP} = \frac{1}{|\Omega|} \sum_{\mathbf{x} \in \Omega} T(\mathbf{x}).$$

For N sampled spins:

$$\widehat{\text{RTP}} = \frac{1}{N} \sum_{i=0}^{N-1} T_i.$$

6.13 Configurable Slot RTP

The helper `configure_rtp` applies the configured slot volatility and target RTP. First, it shapes all configured line and scatter payouts:

$$\text{paytable}_{\text{shaped}}(a, c) = \text{paytable}(a, c)^v,$$

$$\text{scatter_pays}_{\text{shaped}}(c) = \text{scatter_pays}(c)^v.$$

Then it scales all shaped payouts:

$$\text{paytable}'(a, c) = \alpha \text{paytable}_{\text{shaped}}(a, c),$$

$$\text{scatter_pays}'(c) = \alpha \text{scatter_pays}_{\text{shaped}}(c),$$

where:

$$\alpha = \frac{\text{target_rtp}}{\text{exact_rtp}(\text{shaped config})}.$$

This should be done once when constructing a game config, not once per spin.

6.14 Code

```
stop_ranges = [range(len(reel)) for reel in config.reels]
for stops in product(*stop_ranges):
    total_multiplier += evaluate_stops(config, stops).total_multiplier
return total_multiplier / combinations

def configure_rtp(
    config: SlotConfig,
    target: Optional[float] = None,
    volatility: Optional[float] = None,
    max_combinations: Optional[int] = None,
) -> SlotConfig:
    target = target if target is not None else target_rtp("slots", 0.96)
    shaped_config = shape_volatility(config, volatility_exponent(volatility))
    base_rtp = exact_rtp(shaped_config, max_combinations=max_combinations)
    return scale_payouts(shaped_config, target / base_rtp)
```

7 Baccarat: baccarat.py

7.1 Card Values

$$v(A) = 1, \quad v(2), \dots, v(9) = 2, \dots, 9,$$

$$v(10) = v(J) = v(Q) = v(K) = 0.$$

The total is:

$$H = \left(\sum_{c \in \text{hand}} v(c) \right) \bmod 10.$$

7.2 Code

```
def baccarat_card_value(card: Card) -> int:
    if card.rank == "A":
        return 1
    if card.rank in {"10", "J", "Q", "K"}:
        return 0
    return int(card.rank)

def hand_total(cards: tuple[Card, ...]) -> int:
    return sum(baccarat_card_value(card) for card in cards) % 10
```

7.3 Initial Deal

For shuffled shoe D :

$$\text{Player} = (D_0, D_2), \quad \text{Banker} = (D_1, D_3).$$

A natural occurs if:

$$H_P \in \{8, 9\} \quad \text{or} \quad H_B \in \{8, 9\}.$$

If there is a natural, no third cards are drawn.

7.4 Code

```
player_hand = [deck[0], deck[2]]
banker_hand = [deck[1], deck[3]]
natural = player_total in {8, 9} or banker_total in {8, 9}
```

7.5 Draw Rules

If no natural occurs, the player draws if:

$$H_P \leq 5.$$

If the player did not draw, the banker draws if:

$$H_B \leq 5.$$

If the player did draw, let t be the value of the player's third card:

H_B	Banker draws when
0, 1, 2	always
3	$t \neq 8$
4	$2 \leq t \leq 7$
5	$4 \leq t \leq 7$
6	$6 \leq t \leq 7$
7	never

7.6 Code

```
if player_third_card is None:
    return banker_total <= 5
if banker_total <= 2:
    return True
if banker_total == 3:
    return player_third_value != 8
if banker_total == 4:
    return 2 <= player_third_value <= 7
if banker_total == 5:
    return 4 <= player_third_value <= 7
if banker_total == 6:
    return 6 <= player_third_value <= 7
return False
```

7.7 Outcome and Settlement

$$\text{outcome} = \begin{cases} \text{player}, & H_P > H_B, \\ \text{banker}, & H_B > H_P, \\ \text{tie}, & H_P = H_B. \end{cases}$$

Let stake be B , banker commission be γ , and tie payout be τ . Profit is:

$$\begin{aligned} \text{profit}_{\text{player bet}} &= \begin{cases} B, & \text{player wins,} \\ 0, & \text{tie,} \\ -B, & \text{banker wins,} \end{cases} \\ \text{profit}_{\text{banker bet}} &= \begin{cases} B(1 - \gamma), & \text{banker wins,} \\ 0, & \text{tie,} \\ -B, & \text{player wins,} \end{cases} \\ \text{profit}_{\text{tie bet}} &= \begin{cases} B\tau, & \text{tie,} \\ -B, & \text{otherwise.} \end{cases} \end{aligned}$$

Returned payout is:

$$\text{payout} = B + \text{profit}.$$

7.8 Code

```
if bet_on == "tie":
    profit = stake * tie_payout if round_result.outcome == "tie" else -stake
elif round_result.outcome == "tie":
    profit = 0.0
elif bet_on == round_result.outcome:
    multiplier = 1.0 - banker_commission if bet_on == "banker" else 1.0
    profit = stake * multiplier
else:
    profit = -stake
```

8 Blackjack: blackjack.py

8.1 Card Values and Aces

$$v(2), \dots, v(10) = 2, \dots, 10,$$

$$v(J) = v(Q) = v(K) = 10, \quad v(A) = 11 \text{ initially.}$$

If the total T is above 21 and there are aces valued as 11, each ace conversion subtracts 10:

$$T \leftarrow T - 10.$$

The hand is soft if at least one ace remains valued as 11.

8.2 Code

```
total = sum(blackjack_card_value(card) for card in cards)
aces = sum(card.rank == "A" for card in cards)
while total > 21 and aces:
    total -= 10
    aces -= 1
return total, aces > 0
```

8.3 Natural Blackjack

$$\text{natural} \iff |\text{cards}| = 2 \wedge T = 21 \wedge \neg \text{from_split}.$$

8.4 Code

```
def is_natural_blackjack(hand: BlackjackHand) -> bool:
    total, _ = hand_value(hand.cards)
    return len(hand.cards) == 2 and total == 21 and not hand.from_split
```

8.5 Initial Deal

For shuffled deck D :

$$\text{Player} = (D_0, D_2), \quad \text{Dealer} = (D_1, D_3).$$

8.6 Code

```
player_hand = BlackjackHand(cards=(deck[0], deck[2]))
dealer_hand = BlackjackHand(cards=(deck[1], deck[3]))
```

8.7 Actions

Hit appends the next card:

$$H' = H \parallel D_i.$$

Stand marks the hand complete. Double down doubles the bet, draws one card, and stands:

$$B' = 2B.$$

A split is allowed when the two cards have equal blackjack value:

$$v(c_0) = v(c_1).$$

The split creates:

$$H_1 = (c_0, D_i), \quad H_2 = (c_1, D_{i+1}).$$

8.8 Code

```
updated_hand = replace(  
    hand,  
    cards=hand.cards + (card,),  
    bet=hand.bet * 2,  
    stood=True,  
    doubled=True,  
)
```

8.9 Dealer Rule

The dealer draws while:

$$T_D < 17.$$

If the rules say dealer hits soft 17, the dealer also draws when:

$$T_D = 17 \quad \text{and} \quad \text{soft} = \text{true}.$$

8.10 Code

```
def should_dealer_draw(dealer: BlackjackHand, rules: BlackjackRules) -> bool:  
    total, soft = hand_value(dealer.cards)  
    if total < 17:  
        return True  
    return total == 17 and soft and rules.dealer_hits_soft_17
```

8.11 Settlement

Let player total be T_P , dealer total be T_D , hand bet be B , and blackjack payout be $\beta = 1.5$ by default. Net profit is:

$$\text{profit} = \begin{cases} -B, & T_P > 21, \\ 0, & \text{player natural and dealer natural}, \\ B\beta, & \text{player natural only}, \\ -B, & \text{dealer natural only}, \\ B, & T_D > 21, \\ B, & T_P > T_D, \\ -B, & T_P < T_D, \\ 0, & T_P = T_D. \end{cases}$$

Returned payout is:

$$\text{payout} = B + \text{profit}.$$

8.12 Code

```
if player_total > 21:
    profit = -hand.bet
elif player_blackjack and dealer_blackjack:
    profit = 0.0
elif player_blackjack:
    profit = hand.bet * rules.blackjack_payout
elif dealer_blackjack:
    profit = -hand.bet
elif dealer_bust:
    profit = hand.bet
elif player_total > dealer_total:
    profit = hand.bet
elif player_total < dealer_total:
    profit = -hand.bet
else:
    profit = 0.0
```

9 Expected Value and RTP

All games can be interpreted through expected value. If outcome o has probability $P(o)$ and payout multiplier $m(o)$, then:

$$\text{RTP} = \mathbb{E}[m] = \sum_{o \in \Omega} P(o)m(o).$$

House edge is:

$$\text{house edge} = 1 - \text{RTP}.$$

For slots, Ω is the set of reel-stop combinations. For baccarat and blackjack, Ω is the set of possible shuffled deck orders and player decisions. Baccarat has fixed decisions after the shuffle; blackjack is strategy-dependent.

10 Math Audit Notes

- The shared PRNG uses rejection sampling, so bounded integers are unbiased as long as HMAC-SHA256 is modeled as uniformly random.
- Fisher-Yates is uniform because each swap index is drawn uniformly from $\{0, \dots, i\}$.
- With multiple decks, identical rank-suit labels repeat, but the underlying shoe positions are still shuffled uniformly.
- The crash instant-crash branch is approximately $1/25$, with the exact finite 32-bit probability shown above.
- The crash multiplier can return 1.00 through both the explicit divisible-by-25 branch and the $\max(1, \cdot)$ clamp.
- Slot RTP in this module is represented as an average payout multiplier. To express it as a percent, multiply by 100.
- Baccarat settlement uses net profit. A winning tie bet at 8:1 has profit $8B$ and returned payout $9B$.
- Blackjack settlement also uses net profit. A natural blackjack at 3:2 has profit $1.5B$ and returned payout $2.5B$.

11 Verification Flow

A verification-friendly round has this structure:

1. Casino chooses private server seed S .
2. Casino publishes commitment $C_S = \text{SHA256}(S)$.
3. Player chooses or accepts client seed C .
4. Game uses nonce n .
5. Outcome is computed deterministically from (S, C, n) .
6. Casino reveals S .
7. Player checks $\text{SHA256}(S) = C_S$, then recomputes the game outcome.

This proves the revealed seed matches the prior commitment. It does not prove that a payable is favorable or that a game has a high RTP; it proves that the outcome was generated from the committed seed, client seed, and nonce.